ResponseFlow API

Implementation & Deployment

Submission Links

GitHub Repository: https://github.com/tturne21/responseflow

Live API (Render): https://responseflow-api.onrender.com

Swagger Docs: https://responseflow-api.onrender.com/api-docs

Seed Login Credentials

Use these to log in and get a JWT token for testing.

Role	Email	Password
Admin	`admin@responseflow.com`	`Admin1234!`
Manager	`manager@responseflow.com`	`Manager1234!`
Staff	`staff@responseflow.com`	`Staff1234!`
Staff 2	`staff2@responseflow.com`	`Staff1234!`

How to Use Swagger UI

- Go to https://responseflow-api.onrender.com/api-docs in your browser.
- To test a protected endpoint, first run POST /api/auth/login below.
- Copy the token value from the response.
- Click the green Authorize button at the top right of Swagger.
- Paste the token into the Value field, click Authorize, then Close.
- All protected endpoints will now send your token automatically.
- To switch users, click Authorize again, click Logout, then paste a new token.

Testing Plan

Section 1 — Authentication (No token needed)

Signup — Success

POST `/api/auth/signup`

- Click Try it out
- Paste into request body:
  - *{ "name": "Test User", "email": "new@test.com", "password": "Test1234!", "role": "staff" }*
- Click Execute

Expected: 201 Created — Response includes token and user object

Signup — Duplicate Email

POST `/api/auth/signup`

- Click Try it out
- Use the same email: new@test.com
- Click Execute

Expected: 409 Conflict — Email already in use

Signup — Missing Required Field

POST `/api/auth/signup`

- Click Try it out
- Send body without the name field:
  - *{ "email": "noname@test.com", "password": "Test1234!" }*
- Click Execute

Expected: 400 Bad Request — name is required

Login — Success (Admin)

POST `/api/auth/login`

- Click Try it out
- Paste into request body:
  - *{ "email": "admin@responseflow.com", "password": "Admin1234!" }*
- Click Execute
- Copy the token from the response — paste it into Swagger Authorize

Expected: 200 OK — Response includes token and user with role: admin

Login — Wrong Password

POST `/api/auth/login`

- Click Try it out
- Paste into request body:
  - *{ "email": "admin@responseflow.com", "password": "wrongpassword" }*

- Click Execute

Expected: 401 Unauthorized — Invalid email or password

Login — Missing Field

`POST /api/auth/login`

- Click Try it out
- Send body with only email, no password:
  - *{ "email": "admin@responseflow.com" }*
- Click Execute

Expected: 400 Bad Request — email and password are required

Section 2 — Customers (Logged in as Admin)

Setup: Login as admin@responseflow.com / Admin1234! and authorize in Swagger.

Get All Customers

`GET /api/customers`

- Click Try it out
- Leave customerType blank
- Click Execute

Expected: 200 OK — Array of 3 customers: John Smith, Sarah Johnson, Mike Davis

Get All Customers — Filtered

`GET /api/customers`

- Click Try it out
- Enter customerType: guest
- Click Execute

Expected: 200 OK — Only customers with customerType: guest

Create Customer

`POST /api/customers`

- Click Try it out
- Paste into request body:
  - *{ "name": "Alice Brown", "contactInfo": "555-9999", "locationLabel": "Table 7", "customerType": "guest" }*
- Click Execute
- Note the id in the response — this is the customer to delete below

Expected: 201 Created — New customer object with id

Get Single Customer

`GET /api/customers/{id}`

- Click Try it out
- Enter id: 1
- Click Execute

Expected: 200 OK — Returns John Smith

Update Customer

`PUT /api/customers/{id}`

- Click Try it out
- Enter id: 1
- Paste into request body: { "locationLabel": "Table 10" }
- Click Execute

Expected: 200 OK — Updated customer with locationLabel: Table 10

Delete Customer — Success

`DELETE /api/customers/{id}`

- Click Try it out
- Enter the id of Alice Brown from the Create step above (not 1, 2, or 3 — those have service requests)
- Click Execute

Expected: 204 No Content — Empty response body

Delete Customer — Invalid ID

`DELETE /api/customers/{id}`

- Click Try it out
- Enter id: -1
- Click Execute

Expected: 400 Bad Request — Invalid ID format

Delete Customer — Not Found

`DELETE /api/customers/{id}`

- Click Try it out
- Enter id: 9999
- Click Execute

Expected: 404 Not Found — Customer not found

Create Customer — Missing Name

`POST /api/customers`

- Click Try it out

- Body: { "contactInfo": "555-0000" }
- Click Execute

Expected: 400 Bad Request — name is required

Create Customer — No Token

- Click Authorize in Swagger, click Logout to clear token, close

`POST /api/customers`

- Click Try it out
- Body: { "name": "Test" }
- Click Execute

Expected: 401 Unauthorized — No token provided

- Re-authorize with admin token before continuing

Create Customer — Staff Forbidden

- Login as staff@responseflow.com / Staff1234! and authorize in Swagger

`POST /api/customers`

- Click Try it out
- Body: { "name": "Test" }
- Click Execute

Expected: 403 Forbidden — Insufficient permissions

- Switch back to admin token before continuing

Section 3 — Service Requests (Logged in as Admin)

Setup: Authorize in Swagger with admin token.

Create Service Request

`POST /api/service-requests`

- Click Try it out
- Paste into request body:
  - *{ "customerId": 1, "requestType": "manager_help", "description": "Needs assistance", "priority": "high" }*
- Click Execute

Expected: 201 Created — New request with status: open

Create Service Request — Customer Not Found

`POST /api/service-requests`

- Click Try it out
- Body: { "customerId": 9999, "requestType": "other" }
- Click Execute

Expected: 404 Not Found — Customer not found

Create Service Request — Missing requestType

`POST /api/service-requests`

- Click Try it out
- Body: { "customerId": 1 }
- Click Execute

Expected: 400 Bad Request — customerId and requestType are required

Get All Service Requests

`GET /api/service-requests`

- Click Try it out, leave filters blank, click Execute

Expected: 200 OK — Array of all requests

Get All — Filter by Status

`GET /api/service-requests`

- Click Try it out
- Enter status: open
- Click Execute

Expected: 200 OK — Only requests with status: open

Get All — Filter by Priority

`GET /api/service-requests`

- Click Try it out
- Enter priority: high
- Click Execute

Expected: 200 OK — Only requests with priority: high

Get Single Service Request

`GET /api/service-requests/{id}`

- Click Try it out
- Enter id: 1
- Click Execute

Expected: 200 OK — Request with customer and assignments included

Get Single — Invalid ID

`GET /api/service-requests/{id}`

- Click Try it out
- Enter id: -1

- Click Execute

Expected: 400 Bad Request — Invalid ID format

Get Single — Not Found

`GET /api/service-requests/{id}`

- Click Try it out
- Enter id: 9999
- Click Execute

Expected: 404 Not Found — Service request not found

Update Status

`PUT /api/service-requests/{id}`

- Click Try it out
- Enter id: 3 (the unassigned open request)
- Body: { "status": "in_progress" }
- Click Execute

Expected: 200 OK — Updated request

Update — Business Rule: Cannot Complete Without Assignment

`PUT /api/service-requests/{id}`

- Click Try it out
- Enter id: 3 (unassigned request)
- Body: { "status": "completed" }
- Click Execute

Expected: 400 Bad Request — Cannot mark as completed without an assignment

Delete Service Request

`DELETE /api/service-requests/{id}`

- Click Try it out
- Create a new service request via POST above and note its id
- Enter that id
- Click Execute

Expected: 204 No Content — Empty response body

Delete Service Request — Invalid ID

`DELETE /api/service-requests/{id}`

- Enter id: -1

Expected: 400 Bad Request — Invalid ID format

Section 4 — Staff Assignments (Logged in as Admin)

Setup: Authorize in Swagger with admin token.

Create Assignment

`POST /api/staff-assignments`

- Click Try it out
- Paste into request body:
  - *{ "serviceRequestId": 3, "staffUserId": 3, "notes": "Handle ASAP" }*
- Click Execute
- Verify: GET /api/service-requests/3 — status should now be "assigned"

Expected: 201 Created — New assignment with assignedAt timestamp

Create Assignment — Cancelled Request

- First cancel request 3: PUT /api/service-requests/3 with body { "status": "cancelled" }

`POST /api/staff-assignments`

- Click Try it out
- Body: { "serviceRequestId": 3, "staffUserId": 3 }
- Click Execute

Expected: 400 Bad Request — Cannot assign a cancelled request

Create Assignment — Missing Fields

`POST /api/staff-assignments`

- Click Try it out
- Body: { "serviceRequestId": 1 } (no staffUserId)
- Click Execute

Expected: 400 Bad Request — serviceRequestId and staffUserId are required

Get All Assignments

`GET /api/staff-assignments`

- Click Try it out, leave filter blank, click Execute

Expected: 200 OK — Array of all assignments

Get All — Filter by Staff

`GET /api/staff-assignments`

- Click Try it out
- Enter staffUserId: 3
- Click Execute

Expected: 200 OK — Only assignments for staff user 3

Get Single Assignment

`GET /api/staff-assignments/{id}`

- Click Try it out
- Enter id: 1
- Click Execute

Expected: 200 OK — Assignment 1 with request and staff user details

Update Assignment — Mark Complete

`PUT /api/staff-assignments/{id}`

- Click Try it out
- Enter id: 1
- Body: { "notes": "Task finished", "markComplete": true }
- Click Execute
- Verify completedAt is set to a current timestamp in the response

Expected: 200 OK — Assignment with completedAt auto-set

Delete Assignment

`DELETE /api/staff-assignments/{id}`

- Click Try it out
- Create a new assignment via POST above and note its id
- Enter that id
- Click Execute

Expected: 204 No Content — Empty response body

Delete Assignment — Invalid ID

`DELETE /api/staff-assignments/{id}`

- Enter id: -1

Expected: 400 Bad Request — Invalid ID format

Delete Assignment — Not Found

`DELETE /api/staff-assignments/{id}`

- Enter id: 9999

Expected: 404 Not Found — Assignment not found

Section 5 — Authorization: Manager Role

Setup: Login as manager@responseflow.com / Manager1234! and authorize in Swagger.

Manager Cannot Delete Customer (Admin Only)

`DELETE /api/customers/{id}`

- Click Try it out
- Enter id: 2
- Click Execute

Expected: 403 Forbidden — Insufficient permissions — admin only

Manager Can Assign Requests

`POST /api/staff-assignments`

- Click Try it out
- Body: { "serviceRequestId": 2, "staffUserId": 3 }
- Click Execute

Expected: 201 Created — New assignment created

Section 6 — Authorization: Staff Role

Setup: Login as staff@responseflow.com / Staff1234! and authorize in Swagger.

Staff Cannot List All Customers

`GET /api/customers`

- Click Try it out
- Click Execute

Expected: 403 Forbidden — Insufficient permissions

Staff Can View Single Customer

`GET /api/customers/{id}`

- Click Try it out
- Enter id: 1
- Click Execute

Expected: 200 OK — Staff are allowed to view individual customer records

Staff Cannot Create Customer

`POST /api/customers`

- Click Try it out
- Body: { "name": "Test" }
- Click Execute

Expected: 403 Forbidden — Insufficient permissions

Staff Cannot List All Assignments

`GET /api/staff-assignments`

- Click Try it out
- Click Execute

Expected: 403 Forbidden — Insufficient permissions

Staff Can View Own Assignment

`GET /api/staff-assignments/{id}`

- Click Try it out
- Enter id: 1 (assignment 1 belongs to staff@responseflow.com)
- Click Execute

Expected: 200 OK — Returns assignment 1

Staff Cannot View Another Staff Assignment

`GET /api/staff-assignments/{id}`

- Click Try it out
- Enter id: 2 (assignment 2 belongs to staff2@responseflow.com, not this user)
- Click Execute

Expected: 403 Forbidden — You can only view your own assignments

Staff Cannot Update Another Staff Assignment

`PUT /api/staff-assignments/{id}`

- Click Try it out
- Enter id: 2
- Body: { "notes": "Trying to update" }
- Click Execute

Expected: 403 Forbidden — You can only update your own assignments

Staff Can Update Own Assignment

`PUT /api/staff-assignments/{id}`

- Click Try it out
- Enter id: 1 (belongs to this staff user)
- Body: { "notes": "Completed", "markComplete": true }
- Click Execute

Expected: 200 OK — Updated with completedAt set

Staff Cannot Delete Assignment

`DELETE /api/staff-assignments/{id}`

- Click Try it out
- Enter id: 1

- Click Execute

Expected: 403 Forbidden — Insufficient permissions

Staff Cannot Delete Service Request

`DELETE /api/service-requests/{id}`

- Click Try it out
- Enter id: 1
- Click Execute

Expected: 403 Forbidden — Insufficient permissions

Section 7 — No Token (401 Tests)

Setup: Click Authorize in Swagger, click Logout to clear token, close.

No Token — Customers

`GET /api/customers/1`

- Click Try it out
- Click Execute

Expected: 401 Unauthorized — No token provided

No Token — Service Requests

`GET /api/service-requests`

- Click Try it out
- Click Execute

Expected: 401 Unauthorized — No token provided

No Token — Staff Assignments

`GET /api/staff-assignments`

- Click Try it out
- Click Execute

Expected: 401 Unauthorized — No token provided